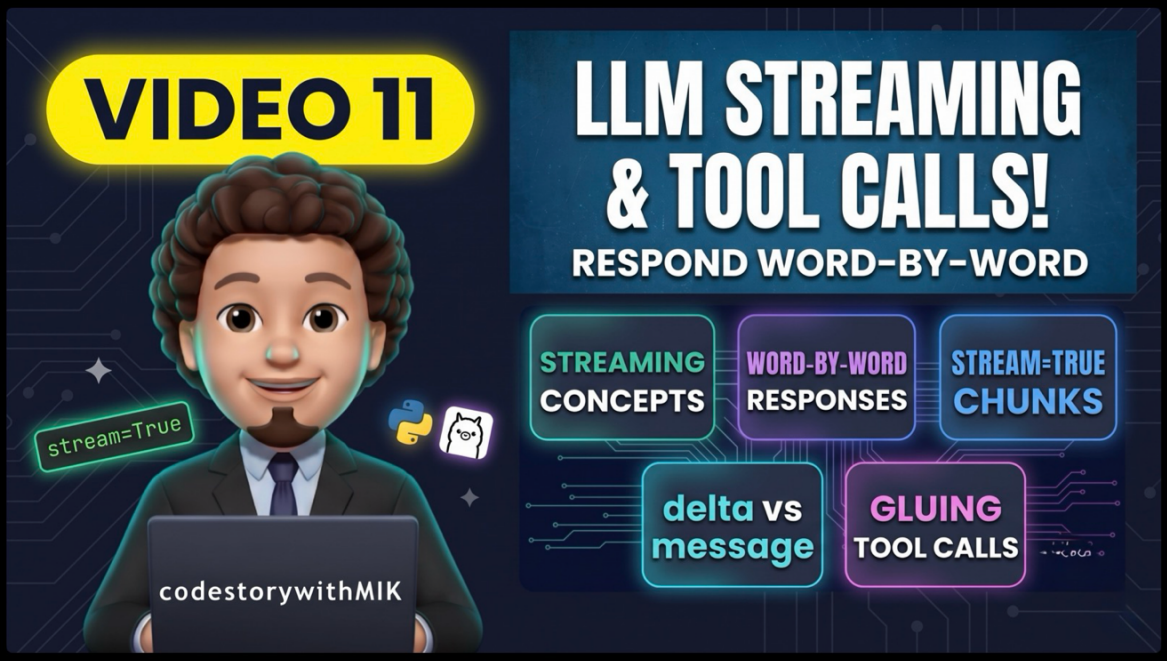
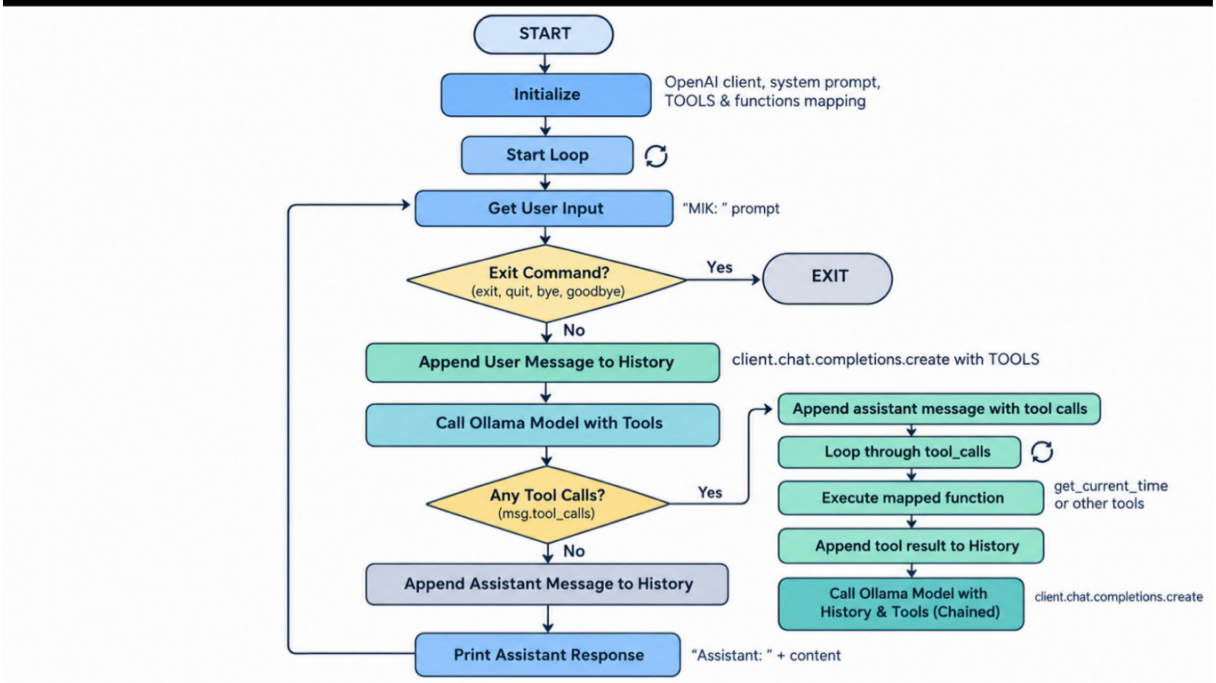


codestorywithMIK

CSwithMIK

codestorywithMIK

Recap : video - 10



- DEBUG mode
- Debuggin colored logs
- Small improvements
- Modular

Let's see a small probl in our AI Agent Is Ch GPT

Our AI Agent → waits or complete response, then print.

ChatGPT → word by word (called streaming)



Our Code Snippet

```python
# Chat loop starts from here
while True:

    user_input = input("\n MIK: ")

    if user_input.lower() in ["exit", "quit", "bye", "goodbye"]:
        break

    messages.append(
        {
            "role": "user",
            "content": user_input
        }
    )

    # Chained Tool Calling
    for step in range(MAX_STEPS):
        response = client.chat.completions.create(
            model=os.getenv("MODEL"),
            messages=messages,
            tools=TOOLS,
        )

        msg = response.choices[0].message

        if not msg.tool_calls:
            break

        log(f"Step {step + 1}: model requested {len(msg.tool_calls)} tool calls")

        messages.append(msg)

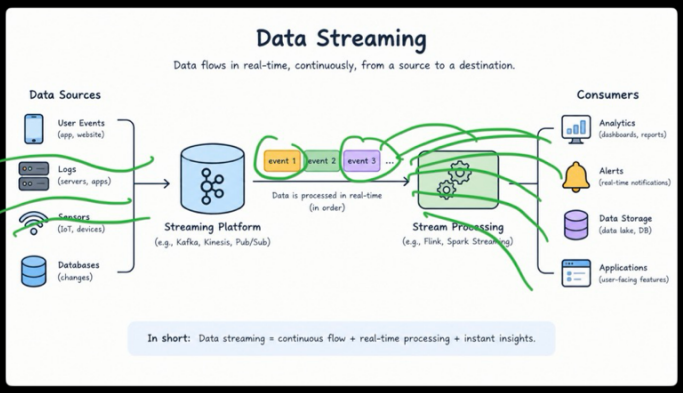
        for tool_call in msg.tool_calls:
```

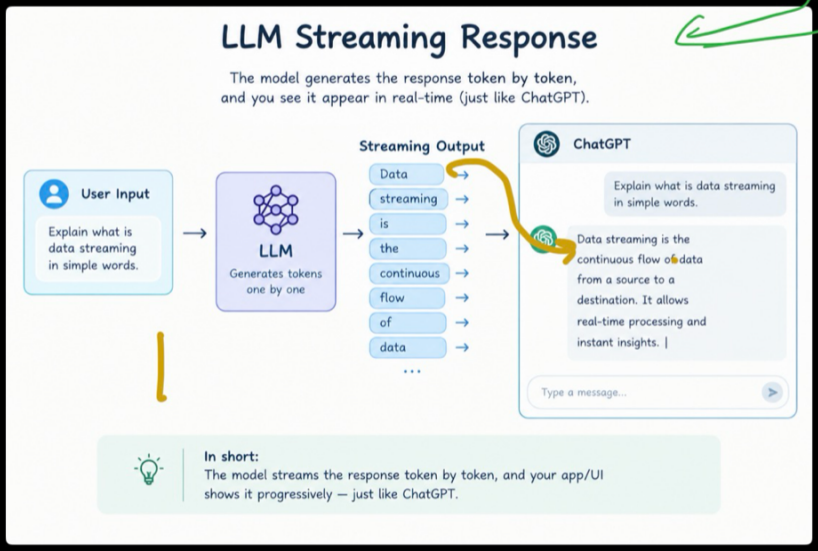
wait

→ Python pauses until the model has finished writing the entire answer. only then we get response.



Streaming


Data Streaming
Data flows in real-time, continuously, from a source to a destination.
Data Sources
User Events
(app, website)
Logs
(servers, apps)
Sensors
(IoT, devices)
Databases
(changes)
Streaming Platform
(e.g., Kafka, Kinesis, Pub/Sub)
event 1
event 2
event 3
...
Data is processed in real-time
(in order)
Stream Processing
(e.g., Flink, Spark Streaming)
Consumers
Analytics
(dashboards, reports)
Alerts
(real-time notifications)
Data Storage
(data lake, DB)
Applications
(user-facing features)
In short: Data streaming = continuous flow + real-time processing + instant insights.


LLM Streaming Response
The model generates the response token by token,
and you see it appear in real-time (just like ChatGPT).
User Input
Explain what is data streaming in simple words.
LLM
Generates tokens one by one
Streaming Output
Data
streaming
is
the
continuous
flow
of
data
...
ChatGPT
Explain what is data streaming in simple words.
Data streaming is the continuous flow of data from a source to a destination. It allows real-time processing and instant insights.
Type a message...
In short:
The model streams the response token by token, and your app/UI shows it progressively — just like ChatGPT.

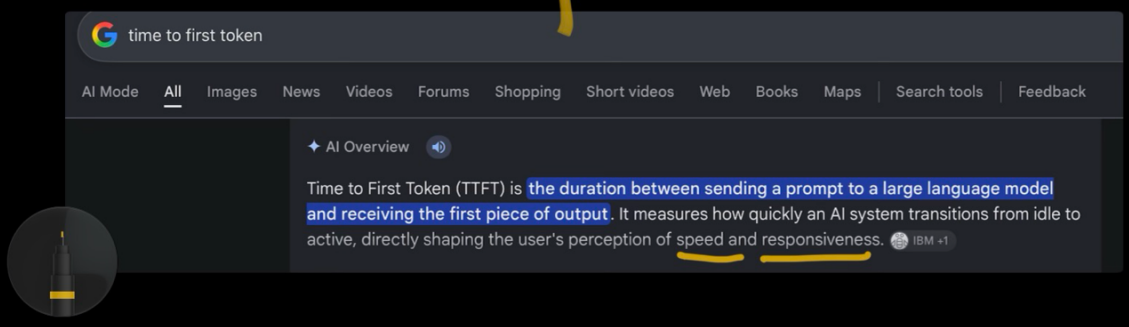
Important catch :

- "Stream" does not make the model faster.
- Total time is still the same
- What improv is the time to see the first word.

waiting 10 seconds to see full response ✗

First word appears in 1-2 seconds (TTFT) ✓

Time To First Token



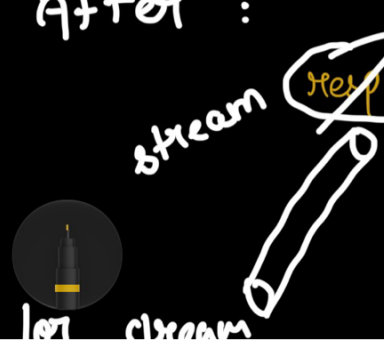
How to make our agent Stre Responses

Before:

```
response = client.chat.completions.create(
    model = os.getenv("MODEL"),
    messages = messages,
    tools = TOOLS
)

response.choices[0].message.content
```

After :




```
response = client.chat.completions.create(
    model = os.getenv("MODEL"),
    messages  messages
    tools  TOOLS
```

lor stream

After :

stream

for stream

response = client.chat.completions.create(

model = os.getenv("MODEL"),

messages messages

tools TOOLS

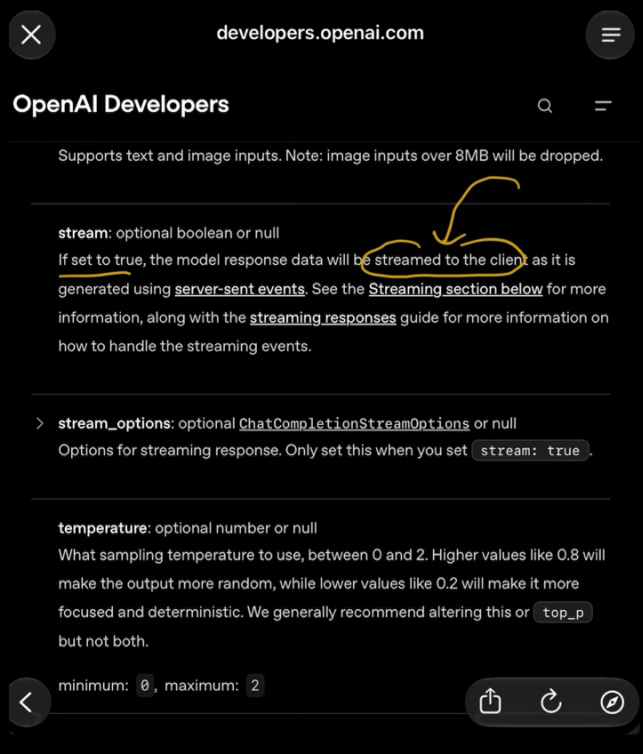
stream True

)

response.choices[0].delta.content

↓

small piece (not full message)



```
stream = client.chat.completions.create(
    model = os.getenv("MODEL"),
    messages = messages
    tools = TOOLS
    stream = True
)
```

chunks

```
for chunk in stream:
    print(chunk.choices[0].delta.content)
```

Let's try it out !!!

① Some chunks have no content. They have other information ⇒ delta.content is None.



qwen3:4b → Thinking model → It streams its thoughts too and they also come

those is too and they also come in chunks and their delta.content is None.

```
for chunk in stream:
    piece = chunk.choices[0].delta.content
    if piece:
        print(piece)
```

Lets fix this.

But, we need to print a response in a line and not line ...



print "Hello") → It prints a new line as well

print ("Hello", end="")

Python keeps them in Buffer and prints in batches (flush = False)

Show as stream

print "Hello", end="", flush=True)

But, we also have entire response in a string full_response for message history.

Keep track of history. So, store the response

What is LLM asks for tool call ??

Recap of current Code

chained tool calling

```python
for step in range(MAX_STEPS):
    response = client.chat.completions.create(
        model=os.getenv("MODEL"),
        messages=messages,
        tools=TOOLS,
    )

    msg = response.choices[0].message

    if not msg.tool_calls:
        break

    log(f"Step {step + 1}: model requested {len(msg.tool_calls)} tool calls")

    messages.append(msg)

    for tool_call in msg.tool_calls:
        result = run_tool(tool_call)
        messages.append({
            "role": "tool",
            "tool_call_id" : tool_call.id,
            "content" : str(result),
        })

else:
    # Ran out of steps: The model never gave the final answer
    fallback = "Sorry, I could not finish this within the step limit."
    messages.append({
        "role": "assistant",
        "content": fallback
    })
    print("🤖 Assistant: " + fallback)
    continue
```

tool-call.id
tool-call.function.name
tool-call.function.arguments



```
stream = client.chat.completions.create(
    model = os.getenv("MODEL"),
    messages = messages  # what's weather in Pune ?
    tools = TOOLS
    stream = True
)
```

```
for chunk in stream :
    delta = chunk.choices[0].delta
    if delta.tool_c
        print(delta.tool_calls)
```

1st Tool Call

```
[ChoiceDeltaToolCall(index=0, id='call_kbs6d0ed',
function=ChoiceDeltaToolCallFunction(arguments='', name='get_city_weather'), type='function')]

[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='{"ci',
name=None), type=None)]

[ChoiceDeltaToolCall(index=0, id=None,
on=ChoiceDeltaToolCallFunction(arguments='ty":"Pu', name=None), type=None)]

eDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='ne"}',
e=None), type=None)]
```

function=ChoiceDeltaToolCallFunction(arguments='ty":"Pu', name=None), type=None)]

[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='ne"}',
name=None), type=None)]

2nd Tool Call

[ChoiceDeltaToolCall(index=1, id='call_kbs6d0ed',
function=ChoiceDeltaToolCallFunction(arguments='', name='get_city_weather'), type='function')]

[ChoiceDeltaToolCall(index=1, id=None, function=ChoiceDeltaToolCallFunction(arguments='{"ci',
name=None), type=None)]

[ChoiceDeltaToolCall(index=1, id=None,
function=ChoiceDeltaToolCallFunction(arguments='ty":"De', name=None), type=None)]

[ChoiceDeltaToolCall(index=1, id=None, function=ChoiceDeltaToolCallFunction(arguments='hi"}',
name=None), type=None)]

"city" : "Delhi"

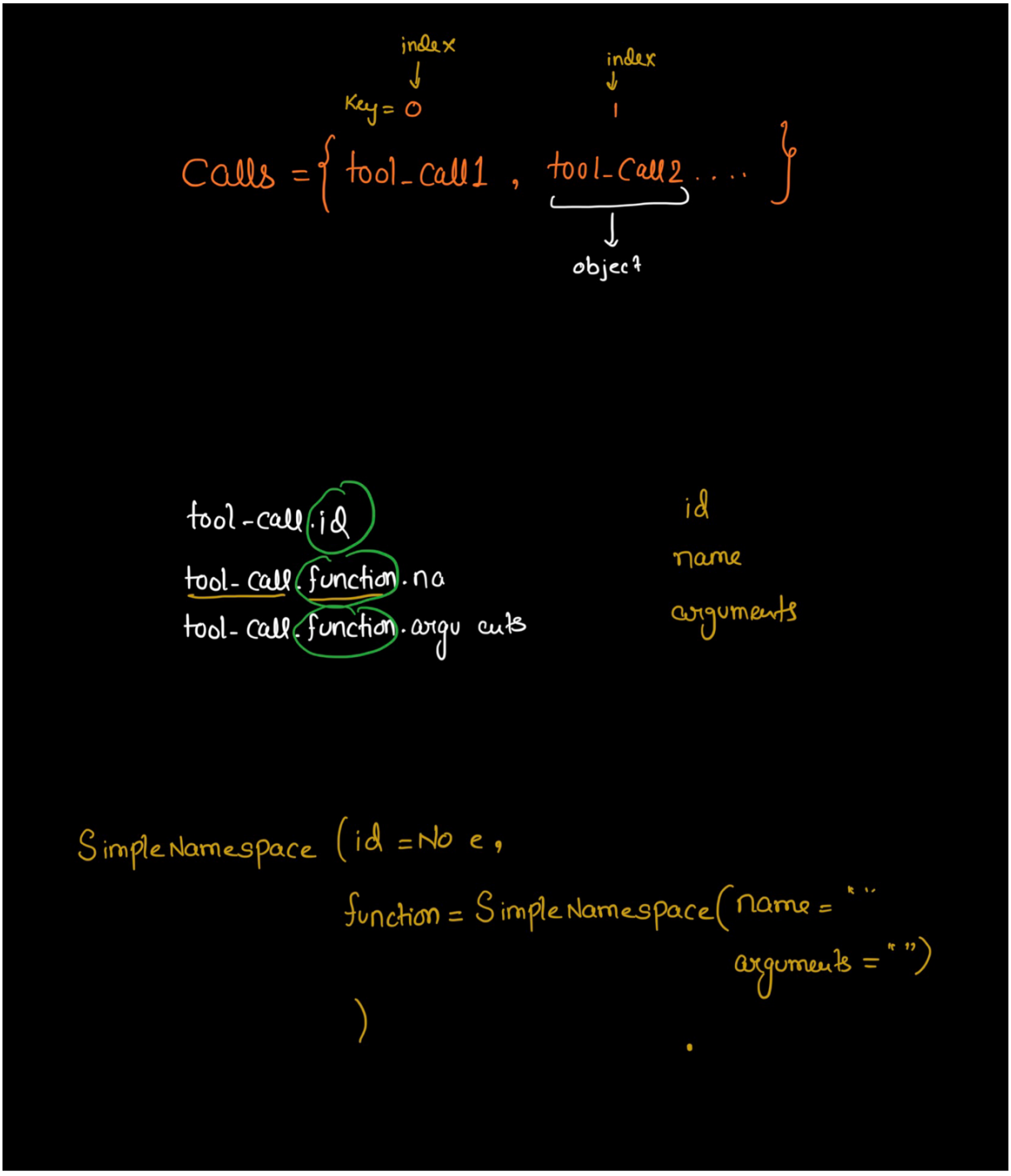
LLM can also send them interleaved ...

[ChoiceDeltaToolCall(index=0, id='call_a', function=ChoiceDeltaToolCallFunction(arguments='',
name='get_city_weather'), type='function')]
[ChoiceDeltaToolCall(index=1, id='call_b', function=ChoiceDeltaToolCallFunction(arguments='',
name='get_city_weather'), type='function')]
[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='{"city":"Pu',
name=None), type=None)]
[ChoiceDeltaToolCall(index=1, id=None, function=ChoiceDeltaToolCallFunction(arguments='{"city":"Del',
name=None), type=None)]
[ChoiceDeltaToolCall(index=0, id=None, function=ChoiceDeltaToolCallFunction(arguments='ne"}',
name=None), type=None)]
[ChoiceDeltaToolCall(index=1, id=None, function=ChoiceDeltaToolCallFunction(arguments='hi"}',
name=None), type=None)]



index

index


index
Key = 0
index
1
Calls = { tool_call1 , tool_call2 . . . }
object
tool-call.id
tool-call.function.na
tool-call.function.argu cuts
id
name
arguments
SimpleNamespace (id = No e ,
function = SimpleNamespace(name = " "
arguments = " ")
)

Improve this code...

```
for chunk in stream:
    if not chunk.choices
        continue

    delta = chunk.choices[0].delta

    for tc in (delta.tool_calls or []):
        obj = c    .setdefault(
            tc.index,
            SimpleNamespace(id=None,
                function=SimpleNamespace(name="",
                                         arguments="")
            )
        )

        if tc.id
            obj.id = tc.id

        if tc.function and tc.function.name:
            obj.function.name += tc.function.name

        if tc.function and tc.function.arguments:
            obj.function.arguments += tc.function.arguments
```

```
calls_list = [ ]
for key in sorted(calls):
    call_list.append(calls[key])
                     tool_call
```

